

Mini-projet : Développement d'un jeu vidéo web basé sur une architecture client-serveur

1. Contexte

Dans le cadre de ce mini-projet, les étudiants devront concevoir et développer un **jeu vidéo web** reposant sur une **architecture client-serveur**. Le projet mettra en œuvre un backend Java exposant des **web services REST** et un frontend web en **JavaScript** consommant ces services.

Le jeu choisi devra rester simple du point de vue graphique, mais suffisamment riche pour illustrer une **logique métier**, la **communication via API REST**, et la **séparation claire entre frontend et backend**.

2. Objectifs pédagogiques

À l'issue de ce mini-projet, l'étudiant devra être capable de : - Concevoir et implémenter une **API REST avec Spring Boot** - Manipuler les échanges **JSON via HTTP** - Consommer des **web services REST en JavaScript** - Mettre en place une **architecture client / serveur** - Implémenter une **logique métier côté serveur** - Utiliser une base de données relationnelle

3. Technologies imposées

- **Backend** : Spring Boot (Java)
 - **Frontend** : HTML5, CSS, JavaScript
 - **Communication** : REST / JSON / HTTP
 - **Base de données** : MySQL ou H2
 - **Outils recommandés** : Maven, Git, Swagger (optionnel)
-

4. Sujet du jeu (au choix)

Les étudiants peuvent choisir **un seul jeu** parmi les propositions suivantes (ou proposer une variante avec validation de l'enseignant) : - Quiz interactif (solo ou multijoueur) - Snake avec sauvegarde des scores - Morpion / Puissance 4 multijoueur - Bataille navale - Jeu d'aventure textuel

5. Fonctionnalités minimales attendues

Backend (Spring Boot)

- API REST pour :
- Gestion des joueurs
- Création et gestion des parties
- Sauvegarde et récupération des scores
- Validation des règles du jeu côté serveur

- Persistance des données (BDD)

Frontend (JavaScript)

- Interface web interactive du jeu
 - Appels REST vers le backend (Fetch ou Axios)
 - Affichage dynamique des données (scores, états de jeu)
 - Gestion des événements utilisateur
-

6. Architecture attendue

- Frontend et backend **séparés**
 - Backend accessible via endpoints REST
 - Données échangées au format JSON
 - Respect des bonnes pratiques REST (GET, POST, PUT, DELETE)
-

7. Version simplifiée (durée 2–3 semaines)

Pour une version courte du projet : - Un seul mode de jeu (solo) - Pas d'authentification complexe - Backend : joueurs + scores uniquement - Frontend : une seule page HTML - Base de données H2

8. Livrables attendus

- Code source du backend (Spring Boot)
 - Code source du frontend (HTML/CSS/JS)
 - Script de création de la base de données
 - Rapport court (5–10 pages) décrivant :
 - L'architecture
 - Les choix techniques
 - Les fonctionnalités réalisées
 - Démonstration du jeu
-

9. Critères d'évaluation (exemple)

Critère	Pondération
Fonctionnalité du jeu	30 %
Qualité de l'API REST	25 %
Frontend JavaScript	20 %
Architecture & propreté du code	15 %
Rapport & présentation	10 %

10. Bonus (optionnel)

- Classement global (leaderboard)
- Authentification (JWT)
- WebSocket pour temps réel
- Documentation Swagger
- Design graphique amélioré

Durée estimée : 2 à 4 semaines

Travail : Individuel ou en binôme